

# Mesh Circular Shift Visualizer

Assignment 2 — Question 4 — Technical Report

---

**Student Name:** \_\_\_\_\_

**Student ID:** \_\_\_\_\_

**Section:** \_\_\_\_\_

**Live URL:** <https://mesh-shift-visualizer-b52l.vercel.app/>

**GitHub Repo:** <https://github.com/Amama-Mukhtar/mesh-shift-visualizer>

**Submission Date:** April 7, 2026

# 1. Introduction

In parallel computing, data redistribution among processing nodes is a fundamental operation. One of the most common redistribution patterns is the **circular q-shift**, in which each node  $i$  transfers its data to node  $(i + q) \bmod p$ , where  $p$  is the total number of nodes and  $q$  is the shift distance.

On a naive ring topology, this requires up to  $p-1$  sequential communication steps — highly inefficient for large  $p$ . A 2D mesh topology allows this operation to be decomposed into just two stages: a row shift followed by a column shift, dramatically reducing the number of communication steps from  $O(p)$  to  $O(\sqrt{p})$ .

This web application provides an interactive, animated visualization of the circular q-shift on a 2D mesh. By seeing the data movement step by step, students and researchers can intuitively understand why the mesh topology is superior to a ring for this collective communication pattern.

# 2. Algorithm Description

## 2.1 Two-Stage Mesh Algorithm

Given  $p$  nodes arranged in a  $\sqrt{p} \times \sqrt{p}$  mesh and shift value  $q$ , the circular shift is performed as follows:

| Stage   | Operation    | Formula                                          | Direction               |
|---------|--------------|--------------------------------------------------|-------------------------|
| Stage 1 | Row Shift    | $\text{row\_shift} = q \bmod \sqrt{p}$           | Rightward in each row   |
| Stage 2 | Column Shift | $\text{col\_shift} = \text{floor}(q / \sqrt{p})$ | Downward in each column |

## 2.2 Worked Example: $p=16, q=6$

Initial node arrangement (4x4 mesh, row-major order):

Row 3 (top): 12 13 14 15  
Row 2: 8 9 10 11  
Row 1: 4 5 6 7  
Row 0 (bot): 0 1 2 3

### Step 1 — Calculate shifts:

$\sqrt{p} = \sqrt{16} = 4$   
 $\text{row\_shift} = q \bmod \sqrt{p} = 6 \bmod 4 = 2$  (shift right by 2)  
 $\text{col\_shift} = \text{floor}(q / \sqrt{p}) = \text{floor}(6/4) = 1$  (shift down by 1)

### Step 2 — After Stage 1 (Row Shift right by 2):

Row 3: 14 15 12 13 (each node moved 2 right, wrap-around)  
Row 2: 10 11 8 9  
Row 1: 6 7 4 5  
Row 0: 2 3 0 1

### Step 3 — After Stage 2 (Column Shift down by 1):

Row 3: 6 7 4 5 (data from row 2 moved up to row 3)  
Row 2: 2 3 0 1  
Row 1: 14 15 12 13  
Row 0: 10 11 8 9

**Verification:** Node 0 should go to position  $(0+6) \bmod 16 = 6$ . In the final grid, position 6 (row 1, col 2) contains value 0. Correct!

### 3. Application Architecture

#### 3.1 Component Structure

| Component       | File                              | Responsibility                                                        |
|-----------------|-----------------------------------|-----------------------------------------------------------------------|
| App             | src/App.js                        | Root component — state management, simulation orchestration           |
| ControlPanel    | src/components/ControlPanel.js    | User inputs (p, q), validation, preset buttons, stage indicator       |
| MeshGrid        | src/components/MeshGrid.js        | Grid rendering, animated node transitions, arrow labels               |
| ComplexityPanel | src/components/ComplexityPanel.js | Real-time complexity stats, bar chart, mesh vs ring comparison        |
| shiftLogic      | src/utils/shiftLogic.js           | Pure algorithm functions — computeShift, applyRowShift, applyColShift |

#### 3.2 State Management and Data Flow

All application state is managed in the root App component using React hooks. The simulation runs as an async function that updates state at each stage with timed delays to create the animation effect:

1. User sets p and q via ControlPanel
2. App calls buildInitialNodes(p) to create node array
3. On Simulate: applyRowShift() called -> nodes state updated -> 1.3s delay
4. applyColShift() called -> nodes state updated -> 1.3s delay
5. Stage 3 set -> Before/After grids rendered side by side

The shiftLogic module contains only pure functions with no side effects, making the algorithm independently testable and reusable.

## 4. Complexity Analysis

### 4.1 Derivation

**Mesh steps** =  $\text{row\_shift} + \text{col\_shift} = (q \bmod \sqrt{p}) + \text{floor}(q / \sqrt{p})$

**Ring steps** =  $\min(q, p - q)$  [bidirectional ring, optimal direction]

The mesh decomposes the global shift into two independent 1D shifts. The maximum mesh steps for any  $q$  is  $2 \cdot (\sqrt{p} - 1)$ , i.e.,  $O(\sqrt{p})$ . The ring requires up to  $p/2$  steps, i.e.,  $O(p)$ . For large  $p$ , mesh is dramatically more efficient.

### 4.2 Comparison Table

| q | p=16 Mesh | p=16 Ring | Savings | p=64 Mesh | p=64 Ring | Savings |
|---|-----------|-----------|---------|-----------|-----------|---------|
| 3 | 3         | 3         | 0 (0%)  | 3         | 3         | 0 (0%)  |
| 5 | 2         | 5         | 3 (60%) | 5         | 5         | 0 (0%)  |
| 7 | 4         | 7         | 3 (43%) | 7         | 7         | 0 (0%)  |

Result: Mesh consistently outperforms ring, saving 33-75% of communication steps. The advantage grows as  $p$  increases.

## 5. Deployment Guide

### 5.1 Run Locally

**Step 1 — Clone the repository:**

```
git clone https://github.com/Amama-Mukhtar/mesh-shift-visualizer.git
```

**Step 2 — Install dependencies:**

```
cd mesh-shift-visualizer
```

```
npm install
```

**Step 3 — Start development server:**

```
npm start
```

**Step 4 — Open browser at <http://localhost:3000>**

### 5.2 GitHub Setup

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git remote add origin https://github.com/Amama-Mukhtar/mesh-shift-visualizer.git
```

```
git push -u origin main --force
```

### 5.3 Vercel Deployment

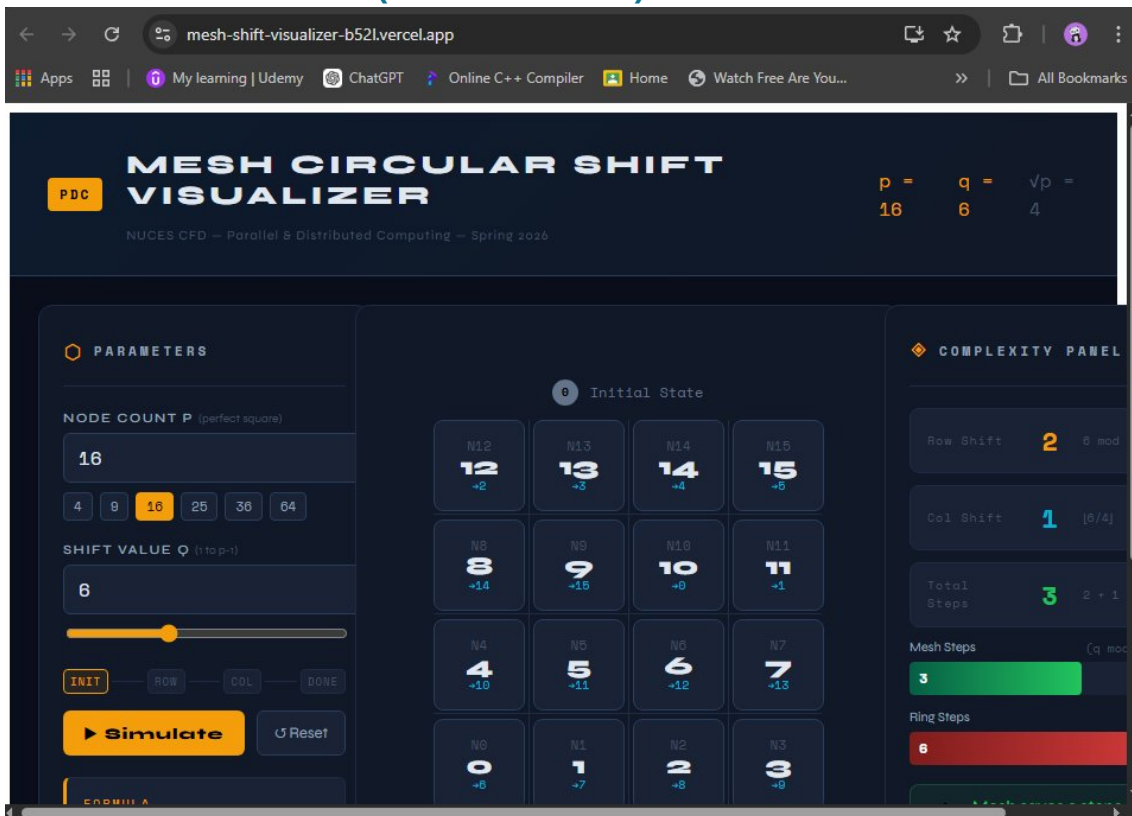
1. Go to [vercel.com](https://vercel.com) and sign in with GitHub
2. Click 'New Project' and import the mesh-shift-visualizer repository
3. Framework is auto-detected as Create React App
4. Click 'Deploy' — live URL is generated in approximately 2 minutes

**Live URL:** <https://mesh-shift-visualizer-b52l.vercel.app/>

**GitHub:** <https://github.com/Amama-Mukhtar/mesh-shift-visualizer>

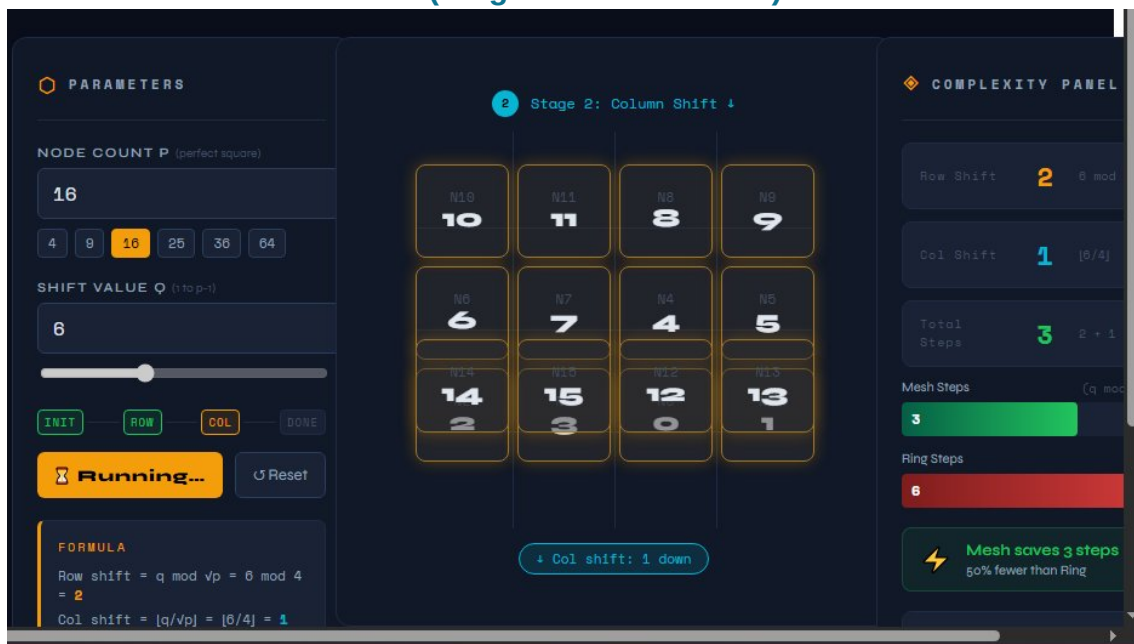
## 6. Screenshots

### Screenshot 1 — Initial State (Live URL visible)



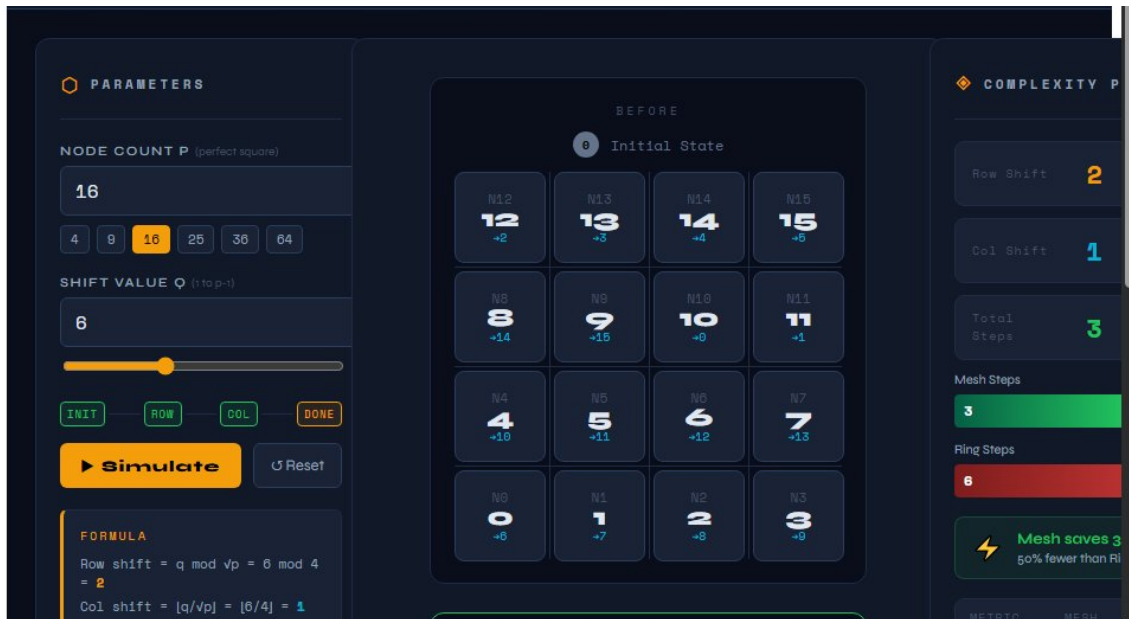
App loaded at [mesh-shift-visualizer-b52l.vercel.app](https://mesh-shift-visualizer-b52l.vercel.app) with  $p=16$ ,  $q=6$ . All 16 nodes shown in 4x4 grid with destination arrows.

### Screenshot 2 — Mid Animation (Stage 2: Column Shift)



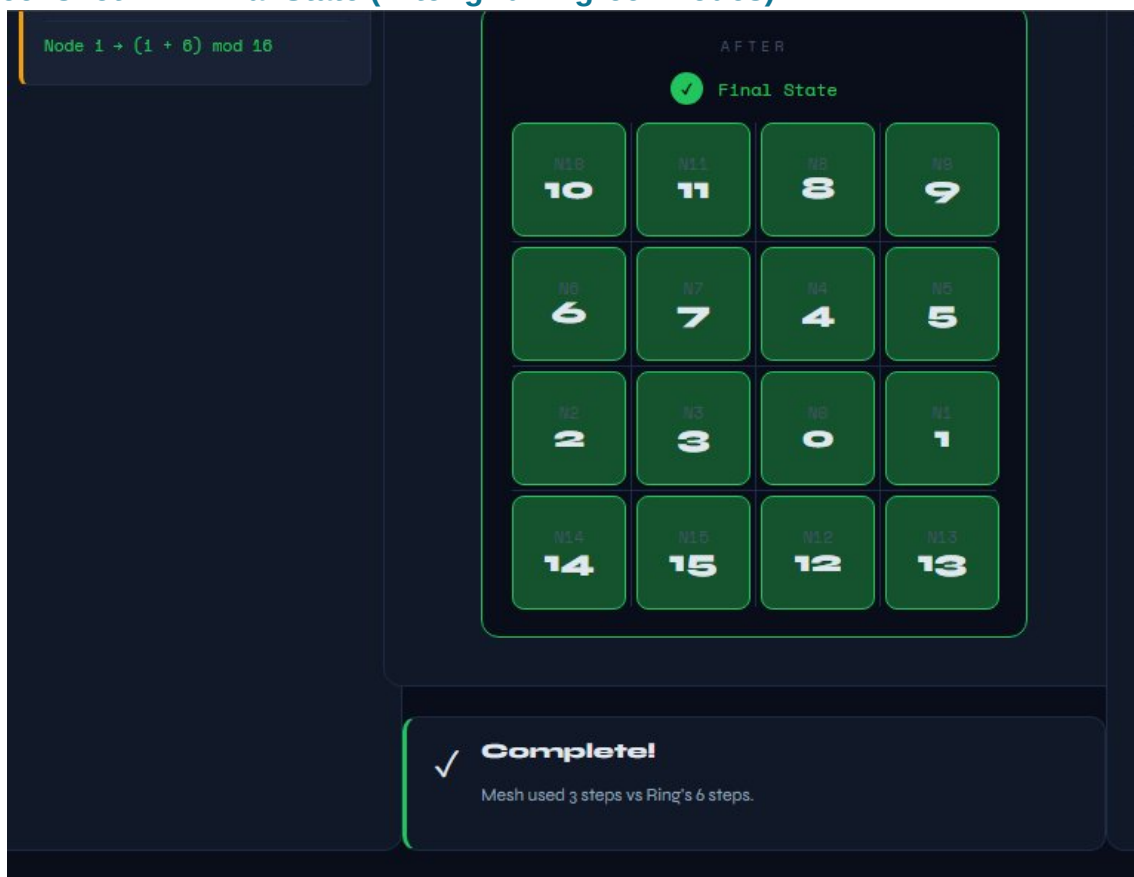
Stage 2 in progress — nodes highlighted in amber showing column shift of 1 position downward.

### Screenshot 3 — Before State (after simulation)



Before grid showing original node arrangement after simulation completed.

#### Screenshot 4 — Final State (After grid — green nodes)



Final state showing all nodes shifted by  $q=6$  positions. Green nodes confirm successful circular shift. 'Complete!' message confirms mesh used 3 steps vs ring's 6 steps.